

操作系统

第二章 进程管理

Unix V6++ 进程图像扩展 和 收缩

同济大学计算机系

Part 1、Unix V6/Unix V6+ + 进程图像扩展和收缩



要解决的问题：扩展虚空间中的堆区 或 用户栈

约束条件：虚地址连续的逻辑页面，物理空间连续存放（连续内存管理方式）

- 用户堆，空间管理：
 - 动态内存分配 malloc。若已分配的用户堆中无足够大空闲块，扩展用户堆，一次3页。
 - 动态内存释放 free。若目标空闲块释放后，用户堆高地址端出现不小于6页的大空闲块，缩小用户堆。
- 用户栈，空间管理：
 - 进程初始栈，1页（4096字节） 4096字节
 - 空间用完，堆栈自动扩展，每次1页（4096字节）

堆顶 == 栈底（虚地址），进程图像扩展失败，OOM，杀死进程

一、进程图像扩展 (堆)

原图像

虚页号 (逻辑页号)	虚空间	虚页号 (逻辑页号)	相对表
0	runtime()	0	null
.....	null null		 null
0x401	第1页代码	0x401	0 U RO P
0x402	第2页代码	0x402	1 U RO P
0x403	第1页数据	0x403	1 U RW P
.....	null		null
.....			
.....			null
.....	null	0x7fe	null
0x7ff	初始栈	0x7ff	2 U RW P
0x800	null..... null		
0xC0000	内核代码/数据	相对表, 逐页描述虚空间中的所有逻辑页: (1) 逻辑属性 (2) 物理空间中的偏移量	
0xC0001			
.....			
.....			
0xC03ff	PPDA	配合 p_addr 和 x_caddr, 可以计算虚实地址映射关系	
..... null	 null		

物理页框号	物理空间
0	内核代码 / 数据
1	
.....	
0x3fe	
.....	
.....	
0x500	第1页代码
0x501	第2页代码
.....	
.....	
0x700	PPDA
0x701	第1页数据
0x702	第1页用户栈
.....	
.....	

堆扩展 (3页)

虚页号 (逻辑页号)	虚空间
0	runtime()
.....	null null
0x401	第1页代码
0x402	第2页代码
0x403	第1页数据
0x404	第2页数据
0x405	第3页数据
0x406	第4页数据
.....	null
0x7ff	初始栈
0x800	null..... null
0xC0000	内核代码/数据
0xC0001	
.....	
.....	
0xC03ff	PPDA
..... null	 null

虚页号 (逻辑页号)	相对表
0	null
.....	 null
0x401	0 U RO P
0x402	1 U RO P
0x403	1 U RW P
0x404	2 U RW P
0x405	3 U RW P
0x406	4 U RW P
.....	null
0x7ff	5 U RW P

Process结构:

- p_addr = 0x725000
- p_size = 0x6000
- 其余字段不变

内存描述符, u_MemoryDescriptor

- m_DataSize = 0x4000
- 其余字段不变

物理页框号	物理空间
0	内核代码 / 数据
1	
.....	
0x3fe	
.....	
.....	
0x500	第1页代码
0x501	第2页代码
.....	
.....	
0x700	PPDA
0x701	第1页数据
0x702	第1页用户栈
.....	
.....	
0x725	PPDA
0x726	第1页数据
0x727	第2页数据
0x728	第3页数据
0x729	第4页数据
0x72a	第1页用户栈
.....	



实现

```
void Process::SBrk()  
{  
    User& u = Kernel::Instance().GetUser();  
    unsigned int newEnd = u.u_arg[0];          // Sbrk() 系统调用的参数：新数据段之后，第1个字节的虚地址  
    MemoryDescriptor& md = u.u_MemoryDescriptor;  
    unsigned int newSize = newEnd - md.m_DataStartAddress;      // 新数据段的尺寸  
  
    if (newEnd == 0)      // 参数为0， Sbrk() 系统调用读取数据段长度  
    {  
        u.u_ar0[User::EAX] = md.m_DataStartAddress + md.m_DataSize;  
        return;  
    }  
  
    if ( false == u.u_MemoryDescriptor.EstablishUserPageTable(md.m_TextStartAddress,  
                                                                md.m_TextSize, md.m_DataStartAddress, newSize, md.m_StackSize) )  
        return;          // 1、重写相对虚实地址映射表
```



```
int change = newSize - md.m_DataSize;
md.m_DataSize = newSize;
newSize += ProcessManager::USIZE + md.m_StackSize;

if ( change < 0 ) // 数据段缩小
    .....
else if ( change > 0 ) // 数据段扩大
{
    // 2、按新尺寸为可交换部分另外分配一块物理内存
    // 3、复制原可交换部分至新区域
    // 4、MapToPageTable( )写系统用户页表映射这块新区域
    this->Expand(newSize);
    // 5、将用户栈平移至新区域底部（高地址端）
    int dst = u.u_procp->p_addr + newSize;
    int count = md.m_StackSize;
    while(count--)
    {
        dst--;
        Utility::CopySeg(dst - change, dst);
    }
}

// 成功。系统调用返回新数据段之后，第1个字节的虚地址
u.u_ar0[User::EAX] = md.m_DataStartAddress + md.m_DataSize;
}
```



```
bool MemoryDescriptor::EstablishUserPageTable( unsigned long textVirtualAddress, unsigned long textSize,
        unsigned long dataVirtualAddress, unsigned long dataSize, unsigned long stackSize )
{
    // 建立相对表
    User& u = Kernel::Instance().GetUser();

    if ( textSize + dataSize + stackSize + PageManager::PAGE_SIZE // 1、检测OOM
        > USER_SPACE_SIZE - textVirtualAddress)
    {
        u.u_error = User::ENOMEM;
        Diagnose::Write("u.u_error = %d\n",u.u_error); // 内核写运行日志
        return false; // Sbrk()系统调用失败返回
    }

    this->ClearUserPageTable(); // 2、把相对表擦干净

    unsigned int phyPageIndex = 0; // 3、为正文段建立相对表表项
    phyPageIndex = this->MapEntry(textVirtualAddress, textSize, phyPageIndex, false);

    phyPageIndex = 1; // 4、为数据段建立相对表表项
    phyPageIndex = this->MapEntry(dataVirtualAddress, dataSize, phyPageIndex, true);
    .....
}
```

重要的辅助函数1



```
bool MemoryDescriptor::EstablishUserPageTable( unsigned long textVirtualAddress, unsigned long textSize,
                                                unsigned long dataVirtualAddress, unsigned long dataSize, unsigned long stackSize )
{
    .....

    unsigned long stackStartAddress =
        (USER_SPACE_START_ADDRESS + USER_SPACE_SIZE - stackSize) & 0xFFFFF000;
    this->MapEntry(stackStartAddress, stackSize, phyPageIndex, true);    // 5、为堆栈段建立相对表表项

    this->MapToPageTable(); // 用新相对表中的信息，更新系统用户页表。注，省掉这步，初始化过不去。
    return true;
}
```




重要的辅助函数 2

```
void Process::Expand(unsigned int newSize) // newSize: 可交换部分, 新尺寸
{
    UserPageManager& userPgMgr = Kernel::Instance().GetUserPageManager(); // 系统用户页表
    .....
    User& u = Kernel::Instance().GetUser(); // 现运行进程的user
    Process* pProcess = u.u_procp; // 的Process结构

    unsigned int oldSize = pProcess->p_size; // 可交换部分, 原尺寸
    pProcess->p_size = newSize; // 1、更新可交换部分尺寸
    unsigned long oldAddress = pProcess->p_addr;
    unsigned long newAddress;

    if ( oldSize >= newSize ) // 尺寸缩小, 释放可交换部分高端物理内存
    {
        if(oldSize > newSize)
            userPgMgr.FreeMemory(oldSize - newSize, oldAddress + newSize);
        return;
    }
    .....
    newAddress = userPgMgr.AllocMemory(newSize); // 2、按新尺寸, 为可交换部分另外分配一块物理内存
    .....
```




```
void Process::Expand(unsigned int newSize)
```

```
{  
    .....  
    if ( NULL == newAddress )  
    {  
        .....  
    }
```

// 内存分配失败，将可交换部分复制到盘交换区
// 得到内存后，跳过后续代码，Expand返回

```
    pProcess->p_addr = newAddress;  
    for ( unsigned int i = 0; i < oldSize; i++ )  
    {  
        Utility::CopySeg(oldAddress + i, newAddress + i);  
    }
```

// 3、更新可交换部分起始物理地址
// 4、复制可交换部分，至新区域

```
    userPgMgr.FreeMemory(oldSize, oldAddress);
```

// 5、释放原先占用的内存区

```
    X86Assembly::CLI();
```

```
    SwtchUStruct(pProcess); // 6、更新核心页表1023#PTE，映射移动后的PPDA区，这里有user和核心栈
```

```
    RetU();
```

```
    X86Assembly::STI();
```

```
    u.u_MemoryDescriptor.MapToPageTable(); // 7、更新用户页表，映射移动后的数据段和用户栈
```

```
}
```



二、进程图像扩展 (栈)

原图像

虚页号 (逻辑页号)	虚空间	虚页号 (逻辑页号)	相对表
0	runtime()	0	null
.....	null null		 null
0x401	第1页代码	0x401	0 U RO P
0x402	第2页代码	0x402	1 U RO P
0x403	第1页数据	0x403	1 U RW P
.....	null		null
.....			
.....			null
.....	null	0x7fe	null
0x7ff	初始栈	0x7ff	2 U RW P
0x800	null..... null		
0xC0000	内核代码/数据	相对表, 逐页描述虚空间中的所有逻辑页: (1) 逻辑属性 (2) 物理空间中的偏移量	
0xC0001			
.....			
.....			
0xC03ff	PPDA	配合 p_addr 和 x_caddr, 可以计算虚实地址映射关系	
..... null	 null		

物理页框号	物理空间
0	内核代码 / 数据
1	
.....	
0x3fe	
.....	
.....	
0x500	第1页代码
0x501	第2页代码
.....	
.....	
0x700	PPDA
0x701	第1页数据
0x702	第1页用户栈
.....	
.....	

栈扩展 (1页)

虚页号 (逻辑页号)	虚空间	虚页号 (逻辑页号)	相对表	物理页框号	物理空间
0	runtime()	0	null	0	内核代码 / 数据
.....	null null		 null	1	
0x401	第1页代码	0x401	0 U RO P		
0x402	第2页代码	0x402	1 U RO P	0x3fe	
0x403	第1页数据	0x403	1 U RW P		
.....	null		null		
.....				0x500	第1页代码
.....			null	0x501	第2页代码
0x7fe	第2页栈	0x7fe	2 U RW P		
0x7ff	第1页栈	0x7ff	3 U RW P		
0x800	null..... null			0x700	PPDA
0xC0000	内核代码/数据	Process结构: • p_addr = 0x725000 • p_size = 0x4000 • 其余字段不变 内存描述符, u_MemoryDescriptor • m_StackSize = 0x2000 • 其余字段不变		0x701	第1页数据
0xC0001				0x702	第1页用户栈
.....					
.....					
0xC03ff	PPDA			0x725	PPDA
..... null	 null			0x726	第1页数据
				0x727	第2页用户栈
				0x728	第1页用户栈
					

实现 1、硬件 MMU抛出缺页异常信号

页级地址映射和内存保护 (1)

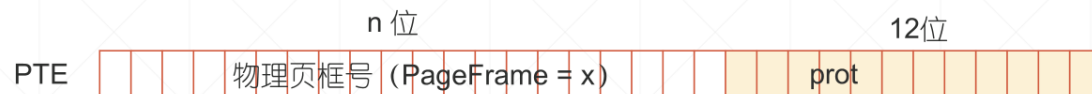


1、查页表

- 用页目录号pagetable查页目录
 - null, 非法内存访问
 - 非空, 映射需要的页表在 base1#物理页框
- 用页号page查页表 (base1#物理页框)
 - null, 非法内存访问
 - 非空, 获得映射所需的页表项



页级地址映射和内存保护 (2)



2、内存保护 (检测prot中的bit)

CS

11

CPU用户态运行

if(CPU用户态运行 & U/S为0)

非法内存访问。应用程序无权访问内核，立即终止现运行进程。

if(写内存单元 & R/W为0)

非法内存访问。访问的单元不可写，终止现运行进程。

if(CPU取指令 & 目标段X位是0, 不可执行)

非法内存访问。修改代码段、只读数据段，终止现运行进程。

MMU

- 将虚地址写入寄存器CR2
- 根据出错原因置错误码 error code:
 - 写了只读页
 - 应用程序访问内核
- 抛出14#缺页异常信号，激活内核，排查错误

页级地址映射和内存保护 (3)





实现

2、内核（缺页异常处理程序） 识别堆栈溢出信号

```
void Exception::PageFault(struct pt_regs* regs, struct pte_context* context)
{
    User& u = Kernel::Instance().GetUser();
    Process* current = u.u_procp;
    MemoryDescriptor& md = u.u_MemoryDescriptor;

    unsigned int cr2;
    __asm__ __volatile__ ("mov %%cr2, %0": "=r"(cr2)); // 1、引发缺页的虚地址

    if( (context->xcs & USER_MODE) == USER_MODE) // 2、缺页发生在用户态，即CPU正在执行应用程序
    {
        if( cr2 < MemoryDescriptor::USER_SPACE_SIZE - md.m_StackSize && cr2 >= context->esp - 8
            && md.m_DataSize + md.m_StackSize + PageManager::PAGE_SIZE <
            MemoryDescriptor::USER_SPACE_SIZE - md.m_DataStartAddress )
        {
            current->SStack(); // 3.1 堆栈扩展
        }
        else
        {
            Diagnose::Write("Invalid MM access"); // 3.2 非法内存访问
            current -> PSignal(User::SIGSEGV); // 现运行进程向自己发信号SIGSEGV（段错误）
            if ( current->IsSig() )
                current->PSig( (pt_context *)&context->eip ); // 响应信号，现运行进程终止
        }
    }
    else // 3、缺页发生在核心态，即CPU正在执行内核
        Utility::Panic("Page Fault in Kernel Mode.");
}
```





实现

```
void Process::SStack()
{
    User& u = Kernel::Instance().GetUser();
    MemoryDescriptor& md = u.u_MemoryDescriptor;
    unsigned int change = 4096;
    md.m_StackSize += change; // 1、登记用户栈的新长度
    unsigned int newSize = ProcessManager::USIZE + md.m_DataSize + md.m_StackSize; // 可交换部分新长度

    if ( false == u.u_MemoryDescriptor.EstablishUserPageTable(md.m_TextStartAddress, // 2、重写相对表
        md.m_TextSize, md.m_DataStartAddress, md.m_DataSize, md.m_StackSize) )
    {
        u.u_error = User::ENOMEM;
        return;
    }

    this->Expand(newSize); // 3、分配新空间、将可交换部分移动至新空间
    .....
}
```



实现

```
void Process::SStack()
{
    .....
    int dst = u.u_procp->p_addr + newSize;
    unsigned int count = md.m_StackSize - change;
    while(count--)
    {
        dst--;
        Utility::CopySeg(dst - change, dst);
    }

    u.u_MemoryDescriptor.MapToPageTable();
}
```

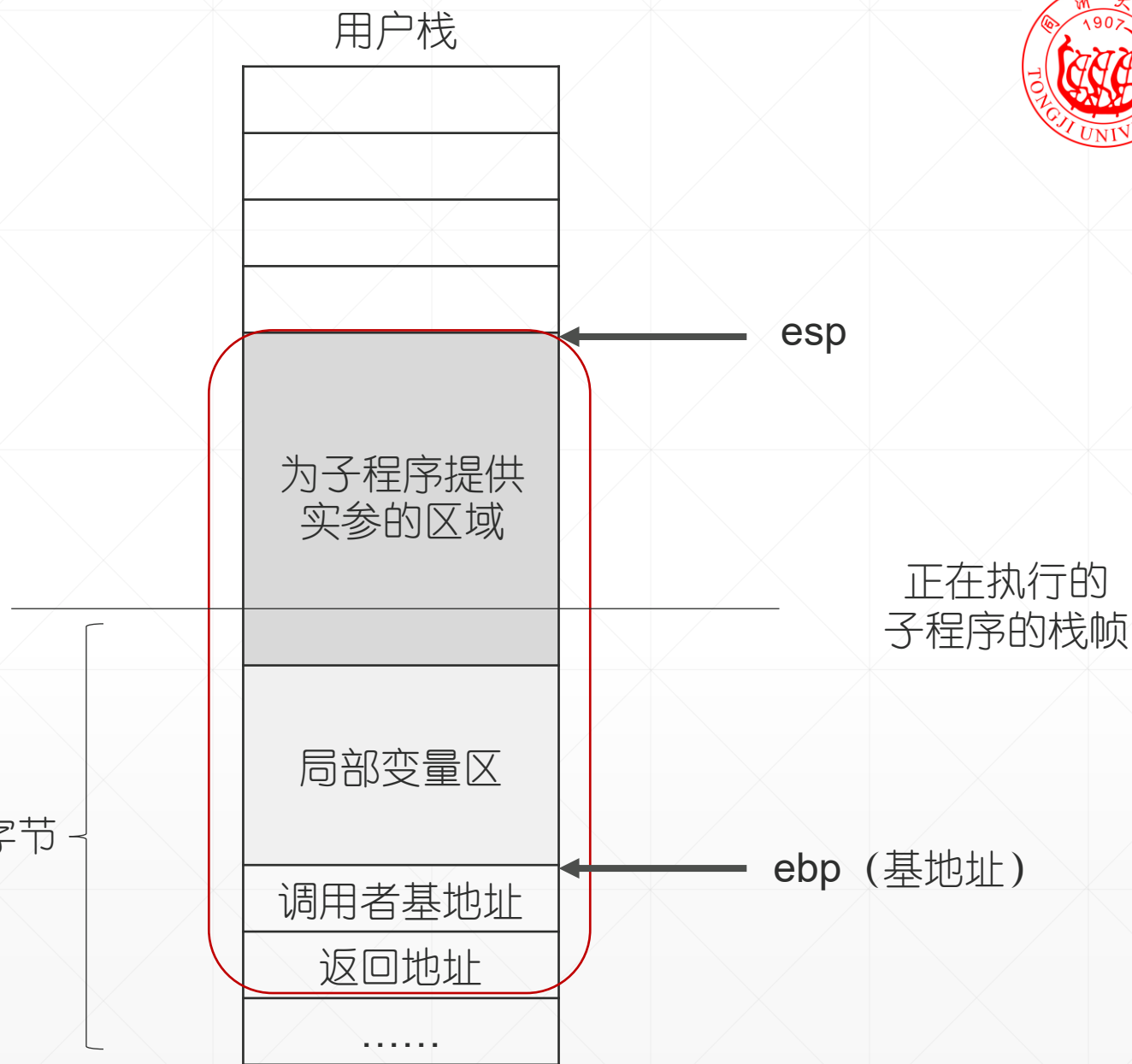
// 4、原来的用户栈下移

// 5、刷新系统用户页表，映射新空间

堆栈扩展，判断条件

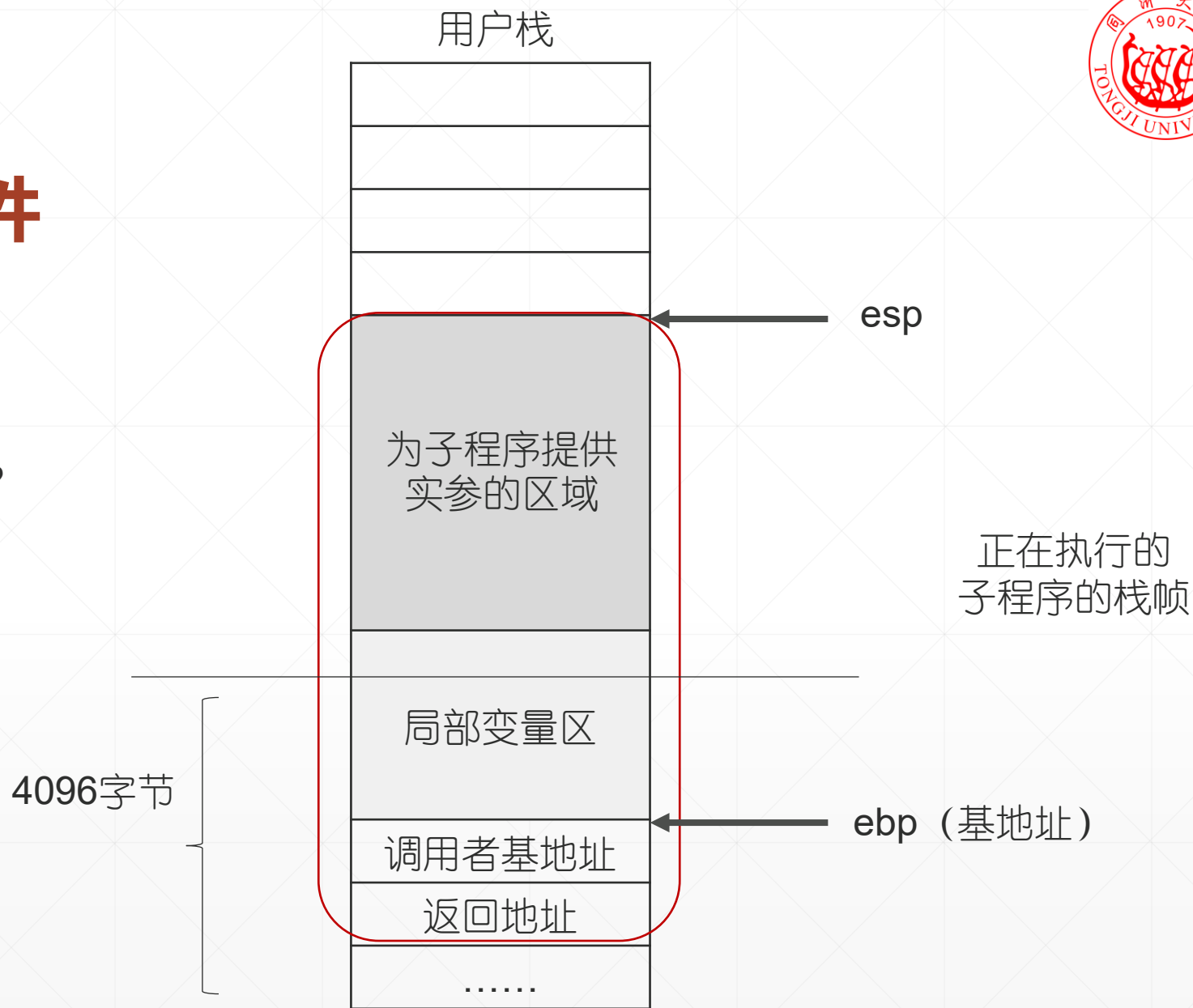
- 1、访问局部变量，MMU会抛出缺页异常嘛？
- 2、为子程序提供实参，~~~~？
- 3、执行call指令，~~~~？
- 4、子程序 push %ebp，MMU会抛出缺页异常吗？
- 5、子程序 访问局部变量，MMU会抛出缺页异常吗？

4096字节



堆栈扩展，判断条件

- 1、构造栈帧阶段，MMU会抛出缺页异常嘛？
- 2、访问局部变量，MMU会抛出缺页异常嘛？





作业1

- 阅读 src/program/stack.c, 观察堆栈扩展过程
- 堆栈扩展, 判断条件是否合理? 给出理由。

```
if( cr2 < MemoryDescriptor::USER_SPACE_SIZE - md.m_StackSize && cr2 >= context->esp - 8
    && md.m_DataSize + md.m_StackSize + PageManager::PAGE_SIZE <
        MemoryDescriptor::USER_SPACE_SIZE - md.m_DataStartAddress )
    current->SStack();
```

// 3.1 堆栈扩展

- $cr2 \geq context \rightarrow esp - 4$, 可不可以?
- $cr2 \geq context \rightarrow esp$, 可不可以?

三、数据段缩短 (3页)

虚页号 (逻辑页号)	虚空间	虚页号 (逻辑页号)	相对表	物理页框号	物理空间
	runtime()	0	null	0	内核代码 / 数据
	null null		 null	1	
	第1页代码	0x401	0 U RO P		
	第2页代码	0x402	1 U RO P	0x3fe	
	第1页数据	0x403	1 U RW P		
	第2页数据	0x404	2 U RW P		
	第3页数据	0x405	3 U RW P	0x500	第1页代码
	第4页数据	0x406	4 U RW P	0x501	第2页代码
	null		null		
	初始栈	0x7ff	5 U RW P		
	null..... null			0x725	PPDA
	内核代码/数据			0x726	第1页数据
				0x727	第2页数据
				0x728	第3页数据
				0x729	第4页数据
	PPDA			0x72a	第1页用户栈
	 null				

虚页号 (逻辑页号)	虚空间	虚页号 (逻辑页号)	相对表	物理页框号	物理空间
	runtime()	0	null	0	内核代码 / 数据
	null null		 null	1	
	第1页代码	0x401	0 U RO P		
	第2页代码	0x402	1 U RO P	0x3fe	
	第1页数据	0x403	1 U RW P		
	null	0x404	null		
	null	0x405	null	0x500	第1页代码
	null	0x406	null	0x501	第2页代码
	null		null		
	初始栈	0x7ff	5 U RW P		
	null..... null			0x725	PPDA
	 null			0x726	第1页数据
				0x727	第2页数据
				0x728	第3页数据
				0x729	第4页数据
				0x72a	第1页用户栈
					

- 1、修改相对表，删除部分PTE置null
- 2、物理内存，用户栈向上平移3页
- 3、free，高地址端释放3个页面



作业2

- 阅读源代码 SBreak()